

Apostila de SQL

Guia Completo — do zero ao avançado

27 capítulos • Exemplos comentados linha a linha

PostgreSQL • MySQL • SQLite • SQL Server

Inclui capítulo dedicado a leitura de dados em blocos:
carregar parte do banco para memória e continuar de onde parou
(paginação, keyset, cursores server-side e streaming).

Sumário

1. Introdução: bancos relacionais e o SQL
2. Ambientes: PostgreSQL, MySQL, SQLite, SQL Server
3. Tipos de dados
4. DDL — CREATE, ALTER, DROP, TRUNCATE
5. Constraints, chaves primárias e estrangeiras
6. DML — INSERT, UPDATE, DELETE, MERGE/UPSERT
7. SELECT — projeção, DISTINCT, alias
8. WHERE — operadores, LIKE, IN, BETWEEN, IS NULL
9. ORDER BY, LIMIT, OFFSET, FETCH
10. Funções: string, numérica, data, condicional

11. Agregações — GROUP BY, HAVING, ROLLUP
12. JOINS — INNER, LEFT, RIGHT, FULL, CROSS, SELF
13. Subqueries e EXISTS
14. CTEs (WITH) e CTEs recursivas
15. Window Functions — OVER, PARTITION BY, ROW_NUMBER, LAG/LEAD
16. Views e Materialized Views
17. Índices — B-Tree, Hash, GIN, GiST, parciais e compostos
18. Transações, ACID e níveis de isolamento
19. Procedures, Functions e Triggers
20. Normalização — 1FN, 2FN, 3FN, BCNF

21. ☐ Ler o banco em blocos: paginação, keyset e cursores
22. Cursores server-side e streaming em Python/Node
23. Performance — EXPLAIN, planos de execução, estatísticas
24. Segurança — SQL Injection, prepared statements, roles
25. JSON e dados semiestruturados no SQL moderno
26. Backup, restore, replicação e migrações
27. Exercícios resolvidos e projetos práticos

1. Introdução: bancos relacionais e o SQL

Um banco de dados relacional organiza informações em tabelas (relações), compostas por linhas (registros) e colunas (atributos). A SQL — Structured Query Language — é a linguagem padrão para criar, consultar e modificar esses dados. Foi criada na IBM na década de 1970 a partir do modelo relacional proposto por Edgar F. Codd.

Existem vários dialetos SQL — PostgreSQL, MySQL, SQL Server, Oracle, SQLite — todos compartilham o núcleo definido pelo padrão ANSI/ISO SQL, mas cada um adiciona extensões próprias. Nesta apostila usaremos principalmente o dialeto PostgreSQL, apontando diferenças relevantes quando necessário.

1.1 Por que aprender SQL?

- É universal: praticamente todo sistema de informação usa um banco relacional em algum ponto.
- É declarativa: você descreve o que quer, não como obter — o otimizador do banco decide o plano.
- É estável: consultas escritas há 20 anos ainda funcionam.
- É poderosa: uma única query bem escrita substitui centenas de linhas de código imperativo.
- É essencial para dados: analistas, engenheiros, cientistas de dados e desenvolvedores back-end usam SQL.

1.2 Estrutura de uma tabela

```
-- Uma tabela é composta de colunas com tipos definidos e linhas com valores.  
-- Exemplo: tabela de clientes de uma loja  
CREATE TABLE clientes (  
    id          BIGSERIAL PRIMARY KEY,          -- identificador único, gerado automaticamente  
    nome        TEXT NOT NULL,                  -- nome do cliente, obrigatório
```

```

    email          TEXT UNIQUE NOT NULL,          -- e-mail obrigatório e único
    nascimento     DATE,                          -- data opcional
    criado_em      TIMESTAMPTZ DEFAULT now()      -- momento de criação (com fuso horário)
);

```

1.3 As grandes categorias de comandos SQL

```

-- DDL (Data Definition Language)    -> estrutura
   : CREATE, ALTER, DROP, TRUNCATE
-- DML (Data Manipulation Language) -> dados
   : INSERT, UPDATE, DELETE, MERGE
-- DQL (Data Query Language)         -> consultas
   : SELECT
-- DCL (Data Control Language)       -> permissões
   : GRANT, REVOKE
-- TCL (Transaction Control)         -> transações
   : BEGIN, COMMIT, ROLLBACK, SAVEPOINT

```

1.4 Modelo lógico x físico

O modelo lógico descreve entidades e relacionamentos (diagrama ER). O modelo físico é a implementação concreta em um SGBD específico — com tipos, índices, particionamento e afinações de performance. Um bom projeto começa pelo modelo lógico e

só depois traduz para o físico.

1.5 Um primeiro exemplo completo

```
-- Criar
CREATE TABLE produtos (
  id      SERIAL PRIMARY KEY,
  nome    TEXT NOT NULL,
  preco   NUMERIC(10,2) NOT NULL CHECK (preco >= 0
),
  estoque INT NOT NULL DEFAULT 0
);

-- Inserir
INSERT INTO produtos (nome, preco, estoque) VALUES
  ('Caneta Azul', 2.50, 100),
  ('Caderno 200p', 19.90, 40),
  ('Mochila', 129.00, 8);

-- Consultar
SELECT nome, preco FROM produtos WHERE estoque >
0 ORDER BY preco DESC;

-- Atualizar
UPDATE produtos SET preco = preco * 1.10 WHERE no
me ILIKE 'mochila%';

-- Remover
DELETE FROM produtos WHERE estoque = 0;
```

Nota: SQL é case-insensitive em palavras-chave (SELECT = select), mas identificadores podem ser sensíveis dependendo do banco. Convenção: palavras-chave em MAIÚSCULAS

e nomes de tabelas/colunas em snake_case.

1.6 O que você vai construir nesta apostila

Ao longo dos capítulos vamos construir gradualmente um sistema fictício de e-commerce com tabelas de clientes, produtos, pedidos, itens_pedido e pagamentos. Esse esquema servirá de base para todos os exemplos, especialmente para os capítulos avançados de JOINS, CTEs, window functions e paginação em blocos.

```
-- Esquema base que usaremos nos próximos capítulos
CREATE TABLE clientes (
  id BIGSERIAL PRIMARY KEY,
  nome TEXT NOT NULL,
  email TEXT UNIQUE NOT NULL,
  cidade TEXT,
  criado_em TIMESTAMPTZ DEFAULT now()
);
CREATE TABLE produtos (
  id BIGSERIAL PRIMARY KEY,
  sku TEXT UNIQUE NOT NULL,
  nome TEXT NOT NULL,
  categoria TEXT NOT NULL,
```

```

    preco NUMERIC(10,2) NOT NULL,
    estoque INT NOT NULL DEFAULT 0
);

CREATE TABLE pedidos (
    id BIGSERIAL PRIMARY KEY,
    cliente_id BIGINT NOT NULL REFERENCES clientes(
id),
    status TEXT NOT NULL DEFAULT 'aberto',
    criado_em TIMESTAMPTZ NOT NULL DEFAULT now(),
    total NUMERIC(12,2) NOT NULL DEFAULT 0
);

CREATE TABLE itens_pedido (
    pedido_id BIGINT NOT NULL REFERENCES pedidos(id
) ON DELETE CASCADE,
    produto_id BIGINT NOT NULL REFERENCES produtos(
id),
    quantidade INT NOT NULL CHECK (quantidade > 0),
    preco_unit NUMERIC(10,2) NOT NULL,
    PRIMARY KEY (pedido_id, produto_id)
);

CREATE INDEX idx_pedidos_cliente ON pedidos(clien
te_id);
CREATE INDEX idx_pedidos_criado ON pedidos(criado
_em DESC);

```

2. Ambientes: PostgreSQL, MySQL, SQLite, SQL Server

Para praticar SQL você precisa de um SGBD instalado (ou de um serviço na nuvem). Cada banco tem particularidades. A seguir, um panorama rápido dos quatro mais populares.

2.1 PostgreSQL

Open source, extensível, com suporte a JSON, arrays, tipos geográficos (PostGIS), índices avançados (GIN, GiST, BRIN) e window functions ricas. É o padrão-ouro para aplicações novas que precisam de robustez.

```
-- Conectar via psql
psql -h localhost -U meu_usuario -d meu_banco
-- Listar bancos, tabelas, descrever
\l          -- lista bancos
\c meu_banco -- conecta
\dt         -- lista tabelas
\d clientes  -- descreve a tabela clientes
\q          -- sai
```

2.2 MySQL / MariaDB

Muito popular em aplicações web. Sintaxe muito parecida com PostgreSQL, mas com diferenças notáveis: usa `AUTO_INCREMENT` em vez de `SERIAL`, aspas de identificador com crase (``nome``) e não tem `RETURNING`.

```
-- Conectar
mysql -h 127.0.0.1 -u root -p
SHOW DATABASES;
USE meu_banco;
SHOW TABLES;
DESCRIBE clientes;
```

2.3 SQLite

Banco embarcado em um único arquivo. Perfeito para prototipagem, testes e apps mobile. Sem servidor, sem configuração — basta abrir o arquivo.

```
sqlite3 loja.db
.tables          -- lista tabelas
.schema clientes -- mostra o CREATE TABLE
.mode column     -- formata saída em colunas
.headers on      -- exibe cabeçalhos
.quit
```

2.4 SQL Server (T-SQL)

Padrão em ambientes Microsoft. Usa TOP em vez de LIMIT, GETDATE() em vez de now(), e tem um subdialeto muito extenso chamado T-SQL.

```
-- Selecionar as 10 primeiras linhas
SELECT TOP 10 * FROM clientes ORDER BY criado_em
DESC;

-- Data atual
SELECT GETDATE();
```

2.5 Tabela de equivalências rápida

Recurso	PostgreSQL	MySQL	SQLite	SQL Server
Auto-incremento	SERIAL/IDENTITY	AUTO_INCREMENT	AUTOINCREMENT	IDENTITY(1,1)
Data/hora agora	now()	NOW()	datetime('now')	GETDATE()
Concatenar strings	ou concat()	CONCAT()		+
Limitar linhas	LIMIT n	LIMIT n	LIMIT n	TOP n / OFFSET-FETCH
Aspas identificador	"nome"	`nome`	"nome"	[nome]
Booleano	BOOLEAN	TINYINT(1)	INTEGER 0/1	BIT
JSON	JSONB (nativo)	JSON	TEXT + JSON1	NVARCHAR + JSON funcs

2.6 Ambientes online para praticar

- db-fiddle.com — cria uma sessão de PostgreSQL, MySQL ou SQLite no navegador.
- sqliteonline.com — SQLite direto no browser, ótimo para exemplos rápidos.

- Supabase / Neon / Railway — Postgres gerenciado na nuvem, camada gratuita generosa.
- Docker — `docker run -e POSTGRES_PASSWORD=pg -p 5432:5432 postgres:16` em segundos.

3. Tipos de dados

Escolher o tipo correto para cada coluna é fundamental: economiza espaço, garante integridade e evita conversões implícitas que estragam a performance.

3.1 Numéricos

```
SMALLINT          -- inteiro 2 bytes  (-32.768 a 32.767)
INTEGER / INT     -- inteiro 4 bytes  (~ ±2,1 bilhões)
BIGINT            -- inteiro 8 bytes  (~ ±9,2 quintilhões) -- use para IDs em tabelas grandes
NUMERIC(p,s)      -- decimal exato   -- use para DINHEIRO
REAL              -- float 4 bytes    -- imprecisão!
DOUBLE PRECISION  -- float 8 bytes    -- imprecisão!
SERIAL/BIGSERIAL  -- integer/bigint auto-incremento (Postgres)
```

Atenção: Nunca use FLOAT/REAL para dinheiro. Erros de arredondamento se acumulam.

Use NUMERIC(10,2).

3.2 Texto

```
CHAR(n)           -- tamanho fixo, preenchido com espaços à direita (raro hoje)
VARCHAR(n)        -- tamanho variável até n caracteres
```

```
TEXT          -- tamanho variável sem limite (no Po
stgres é tão rápido quanto VARCHAR)
UUID          -- 128 bits para identificadores glob
ais
```

3.3 Data e hora

```
DATE          -- só data                '2026-07
-17'
TIME          -- só hora                '15:30:0
0'
TIMESTAMP     -- data + hora (sem fuso)  '2026-07
-17 15:30:00'
TIMESTAMPTZ   -- data + hora COM fuso    '2026-07
-17 15:30:00-03'    << recomendado
INTERVAL      -- duração                '2 hours
30 minutes'
```

Nota: Prefira sempre TIMESTAMPTZ para timestamps. Armazenar sem fuso é fonte infinita de bugs quando a aplicação escala para outros países.

3.4 Booleano

```
BOOLEAN      -- true / false / NULL
-- Em MySQL não há BOOLEAN de verdade: usa-se TIN
YINT(1).
```

3.5 JSON, arrays e tipos especiais (Postgres)

```
JSONB        -- JSON binário indexável (prefer
ir a JSON)
UUID
INET / CIDR   -- endereços IP
```



```
BYTEA          -- bytes binários
ARRAY          -- ex.: TEXT[], INT[] (arrays na
tivos)
tsvector       -- busca full-text
geometry       -- via extensão PostGIS
```

3.6 Conversão de tipos (CAST)

```
SELECT CAST('42' AS INTEGER);          -- SQL padrão
SELECT '42'::INTEGER;                  -- atalho do
Postgres
SELECT CAST(preco AS TEXT) FROM produtos;
SELECT '2026-07-17'::DATE + INTERVAL '30 days';
-- soma 30 dias
```

3.7 NULL — o valor ausente

NULL não é zero nem string vazia — significa desconhecido. Qualquer operação aritmética ou de comparação com NULL retorna NULL:

```
SELECT 10 + NULL;          -- NULL
SELECT 'a' || NULL;        -- NULL
SELECT NULL = NULL;        -- NULL (não é TRUE!)
-- Para testar use IS NULL / IS NOT NULL:
SELECT * FROM clientes WHERE cidade IS NULL;
-- COALESCE devolve o primeiro valor não-NULL
SELECT COALESCE(apelido, nome, 'anônimo') FROM cl
ientes;
-- NULLIF vira NULL quando os dois valores são ig
uais
```

```
SELECT NULLIF(preco, 0) FROM produtos;
```

4. DDL — CREATE, ALTER, DROP, TRUNCATE

A DDL define e altera a estrutura do banco: bancos, esquemas, tabelas, índices, views.

4.1 CREATE DATABASE / SCHEMA

```
CREATE DATABASE loja
  WITH ENCODING = 'UTF8'
      LC_COLLATE = 'pt_BR.UTF-8'
      LC_CTYPE   = 'pt_BR.UTF-8'
      TEMPLATE   = template0;

-- Esquemas ajudam a organizar tabelas por domínio
CREATE SCHEMA vendas;
CREATE SCHEMA financeiro;
CREATE TABLE vendas.pedidos (...);
CREATE TABLE financeiro.faturas (...);
```

4.2 CREATE TABLE completo

```
CREATE TABLE clientes (
  id          BIGSERIAL          PRIMARY KEY,
  nome        TEXT              NOT NULL,
  email       TEXT              NOT NULL UNIQUE,
  cpf         CHAR(11)          UNIQUE,
  idade       INT               CHECK (idade >= 0 AND idade < 150),
  cidade      TEXT              DEFAULT 'São Paulo',
```

```

    ativo          BOOLEAN          NOT NULL DEFAULT TRUE,
    criado_em      TIMESTAMPTZ      NOT NULL DEFAULT now
(),
    metadata       JSONB            DEFAULT '{}'::jsonb
);

```

4.3 ALTER TABLE

```

-- adicionar coluna
ALTER TABLE clientes ADD COLUMN telefone TEXT;

-- remover coluna
ALTER TABLE clientes DROP COLUMN telefone;

-- renomear coluna
ALTER TABLE clientes RENAME COLUMN cidade TO municipio;

-- alterar tipo (pode precisar de USING para conversão)
ALTER TABLE clientes ALTER COLUMN idade TYPE SMALLINT USING idade::smallint;

-- adicionar / remover constraint
ALTER TABLE clientes ADD CONSTRAINT chk_email_arroba CHECK (position('@' in email) > 1);
ALTER TABLE clientes DROP CONSTRAINT chk_email_arroba;

-- renomear tabela
ALTER TABLE clientes RENAME TO usuarios;

```

4.4 DROP e TRUNCATE

```

DROP TABLE IF EXISTS logs;                                -- remove
a tabela e os dados

```

```
DROP TABLE pedidos CASCADE;           -- remove
    também dependências (FKs, views)
TRUNCATE TABLE logs;                   -- apaga
    TODAS as linhas rapidamente
TRUNCATE TABLE logs RESTART IDENTITY;  -- também
    zera o SERIAL
```

Atenção: TRUNCATE não pode ser desfeito fora de transação. Sempre confirme o alvo antes de rodar. Em produção prefira BEGIN; TRUNCATE ...; -- confere -- COMMIT;.

4.5 IF NOT EXISTS e migrações idempotentes

```
-- Comandos que rodam várias vezes sem erro (bom
para migrações versionadas)
CREATE TABLE IF NOT EXISTS clientes (...);
CREATE INDEX IF NOT EXISTS idx_clientes_email ON
clientes(email);
ALTER TABLE clientes ADD COLUMN IF NOT EXISTS tel
efone TEXT;
DROP TABLE IF EXISTS logs_antigos;
```

4.6 Boas práticas de nomenclatura

- Tabelas no plural e em snake_case: clientes, itens_pedido.
- Colunas no singular: nome, preco_unit.
- Chave primária sempre chamada id (evita ambiguidade em JOINS).

- Foreign keys sufixadas por `_id`: `cliente_id`, `produto_id`.
- Índices prefixados por `idx_` e constraints por `chk_`/`uk_`/`fk_`.
- Nada de acentos, espaços ou letras maiúsculas em nomes — evita necessidade de aspas duplas.

5. Constraints, chaves primárias e estrangeiras

Constraints são regras que o banco de dados força a serem verdadeiras. Se você depende só da aplicação para validar, mais cedo ou mais tarde algum job, importação ou desenvolvedor distraído vai corromper os dados.

5.1 PRIMARY KEY

Identifica unicamente cada linha da tabela. Uma tabela pode ter apenas uma PK, mas ela pode ser composta por várias colunas.

```
-- PK simples
CREATE TABLE produtos (id BIGSERIAL PRIMARY KEY,
nome TEXT);

-- PK composta (útil para tabelas de junção)
CREATE TABLE itens_pedido (
    pedido_id BIGINT REFERENCES pedidos(id) ON DEL
ETE CASCADE,
    produto_id BIGINT REFERENCES produtos(id),
    quantidade INT NOT NULL,
    PRIMARY KEY (pedido_id, produto_id)
);
```

5.2 UNIQUE

```
-- Um único email por cliente
ALTER TABLE clientes ADD CONSTRAINT uk_clientes_email UNIQUE (email);

-- Combinação única (evita duas tarifas ativas para o mesmo produto no mesmo mês)
ALTER TABLE tarifas ADD CONSTRAINT uk_tarifa_produto_mes UNIQUE (produto_id, mes_ref);
```

5.3 NOT NULL e DEFAULT

```
ativo BOOLEAN NOT NULL DEFAULT TRUE
criado_em TIMESTAMPTZ NOT NULL DEFAULT now()
```

5.4 CHECK

```
CREATE TABLE pagamentos (
    id BIGSERIAL PRIMARY KEY,
    valor NUMERIC(10,2) NOT NULL CHECK (valor > 0),
    status TEXT NOT NULL CHECK (status IN ('pendente', 'pago', 'estornado')),
    bandeira TEXT CHECK (bandeira ~ '^[A-Z]+$') --
    regex: só letras maiúsculas
);
```

5.5 FOREIGN KEY — integridade referencial

Uma FK garante que o valor de uma coluna existe em outra tabela. Ao alterar ou remover a linha referenciada, a FK pode agir de várias formas:


```

CREATE TABLE pedidos (
  id BIGSERIAL PRIMARY KEY,
  cliente_id BIGINT NOT NULL
    REFERENCES clientes(id)
    ON UPDATE CASCADE      -- se o id do cliente m
udar, atualiza aqui também
    ON DELETE RESTRICT     -- proíbe apagar o clie
nte enquanto houver pedido
);

-- Alternativas para ON DELETE:
--   CASCADE      -> apaga junto
--   SET NULL     -> zera a coluna
--   SET DEFAULT  -> volta ao DEFAULT
--   NO ACTION    -> igual a RESTRICT (padrão)

```

5.6 DEFERRABLE — validar no commit

```

-- Útil para inserir pai e filho na ordem inversa
dentro de uma transação
ALTER TABLE pedidos
  ADD CONSTRAINT fk_pedidos_cliente
  FOREIGN KEY (cliente_id) REFERENCES clientes(id
)
  DEFERRABLE INITIALLY DEFERRED;

```

5.7 Exclusion constraints (Postgres)

```

-- Impede sobreposição de reservas para a mesma s
ala
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE TABLE reservas (
  id BIGSERIAL PRIMARY KEY,

```

```
sala_id INT NOT NULL,  
periodo TSRANGE NOT NULL,  
EXCLUDE USING gist (sala_id WITH =, periodo WITH  
H &&)  
);
```

6. DML — INSERT, UPDATE, DELETE, MERGE/UPSERT

6.1 INSERT

```
-- Forma canônica: listar as colunas
INSERT INTO clientes (nome, email, cidade)
VALUES ('Ana', 'ana@x.com', 'Recife'),
       ('Bruno', 'bruno@x.com', 'Curitiba'),
       ('Carla', 'carla@x.com', 'São Paulo');

-- INSERT ... RETURNING (Postgres) devolve o registro criado
INSERT INTO clientes (nome, email) VALUES ('Diana', 'di@x.com')
RETURNING id, criado_em;

-- INSERT a partir de SELECT
INSERT INTO clientes_backup (id, nome, email)
SELECT id, nome, email FROM clientes WHERE ativo = FALSE;
```

6.2 UPDATE

```
-- Atualização direta
UPDATE produtos SET preco = preco * 1.10 WHERE categoria = 'papelaria';

-- Atualização com JOIN (Postgres: FROM ; SQL Server: FROM/UPDATE-FROM; MySQL: UPDATE ... JOIN)
UPDATE pedidos p
  SET total = sub.total
FROM (
  SELECT pedido_id, SUM(quantidade * preco_unit) AS total
```

```
        FROM itens_pedido GROUP BY pedido_id
    ) sub
    WHERE sub.pedido_id = p.id;
-- Múltiplas colunas
UPDATE clientes
    SET ativo = FALSE,
        inativado_em = now()
    WHERE ultimo_login < now() - INTERVAL '1 year';
```

Atenção: Nunca rode UPDATE sem WHERE em produção. Rode primeiro um SELECT com a mesma cláusula para conferir quantas linhas serão afetadas.

6.3 DELETE

```
DELETE FROM logs WHERE criado_em < now() - INTERV
AL '90 days';
-- DELETE com JOIN (usando USING no Postgres)
DELETE FROM pedidos p
    USING clientes c
    WHERE p.cliente_id = c.id AND c.ativo = FALSE;
-- DELETE com RETURNING para auditar
DELETE FROM sessoes WHERE expira_em < now()
RETURNING id, usuario_id;
```

6.4 UPSERT — INSERT ... ON CONFLICT

Ao inserir uma linha que violaria uma restrição UNIQUE, você pode atualizar em vez de dar erro.

```
INSERT INTO estoque (produto_id, quantidade)
VALUES (42, 10)
ON CONFLICT (produto_id) DO UPDATE
    SET quantidade = estoque.quantidade + EXCLUDED.
quantidade;
-- EXCLUDED referencia a linha que estava sendo i
nserida.
-- Ignorar duplicatas silenciosamente:
INSERT INTO tags (nome) VALUES ('sql') ON CONFLIC
T DO NOTHING;
```

6.5 MERGE (SQL padrão / SQL Server / Postgres 15+)

```
MERGE INTO estoque e
USING (SELECT 42 AS produto_id, 10 AS qtd) s
    ON e.produto_id = s.produto_id
WHEN MATCHED THEN
    UPDATE SET quantidade = e.quantidade + s.qtd
WHEN NOT MATCHED THEN
    INSERT (produto_id, quantidade) VALUES (s.produ
to_id, s.qtd);
```

6.6 Inserindo grandes volumes (bulk load)

```
-- Postgres: COPY é dezenas de vezes mais rápido
que INSERT em loop
COPY clientes (nome, email, cidade) FROM '/tmp/cl
ientes.csv' CSV HEADER;
-- Ou pelo lado do cliente:
\copy clientes FROM 'clientes.csv' CSV HEADER;
-- MySQL:
```

```
LOAD DATA LOCAL INFILE 'clientes.csv' INTO TABLE
clientes
FIELDS TERMINATED BY ',' ENCLOSED BY '"' IGNORE 1
LINES;
```

7. SELECT — projeção, DISTINCT, alias

7.1 Anatomia da consulta SELECT

```
SELECT    [DISTINCT] <expressões>
  FROM    <tabelas / joins>
  [WHERE  <filtros de linha>]
  [GROUP BY <colunas>]
  [HAVING <filtros após agregação>]
  [ORDER BY <colunas> [ASC|DESC]]
  [LIMIT n] [OFFSET m];
```

A ordem lógica de execução, no entanto, é diferente da ordem sintática:

```
1. FROM / JOIN      -- monta o produto entre tabelas
2. WHERE            -- filtra linhas
3. GROUP BY         -- agrupa
4. HAVING           -- filtra grupos
5. SELECT           -- projeta as colunas
6. DISTINCT
7. ORDER BY
8. LIMIT / OFFSET
```

Por isso não se pode usar um alias definido no SELECT dentro do WHERE — o WHERE roda antes.

7.2 Projeção e alias

```
SELECT id,  
       nome AS cliente,  
       upper(cidade) AS cidade_upper,  
       nascimento,  
       age(nascimento) AS idade      -- alias com "  
AS" (opcional)  
FROM clientes;  
  
-- Alias de tabela é obrigatório para tornar JOIN  
s legíveis:  
SELECT c.nome, p.id AS pedido_id  
FROM clientes c  
JOIN pedidos p ON p.cliente_id = c.id;
```

7.3 DISTINCT

```
-- Cidades únicas dos clientes  
SELECT DISTINCT cidade FROM clientes;  
  
-- DISTINCT ON (Postgres): pega a primeira linha  
por chave  
SELECT DISTINCT ON (cliente_id) cliente_id, id AS  
pedido_id, criado_em  
FROM pedidos  
ORDER BY cliente_id, criado_em DESC;  -- último  
pedido de cada cliente
```

7.4 Expressões e cálculos

```
SELECT id,  
       preco,  
       estoque,  
       preco * estoque AS valor_em_estoque,
```



```
CASE WHEN estoque = 0 THEN 'esgotado'
      WHEN estoque < 5 THEN 'baixo'
      ELSE 'ok'
END AS status_estoque
FROM produtos;
```

7.5 SELECT sem FROM

```
SELECT now(), current_user, version();
SELECT 2 + 2 AS resposta;
```

8. WHERE — operadores, LIKE, IN, BETWEEN, IS NULL

8.1 Comparações básicas

```
SELECT * FROM produtos WHERE preco = 19.90;  
SELECT * FROM produtos WHERE preco <> 0;  
-- diferente  
SELECT * FROM produtos WHERE preco >= 100;  
SELECT * FROM clientes WHERE cidade = 'Recife';
```

8.2 AND, OR, NOT e parênteses

```
SELECT * FROM produtos  
  WHERE (categoria = 'papelaria' OR categoria = 'e  
scritório')  
    AND estoque > 0  
    AND preco <= 50  
    AND NOT descontinuado;
```

Nota: Sem parênteses, AND tem precedência sobre OR. Sempre agrupe explicitamente para evitar bugs sutis.

8.3 IN e NOT IN

```
SELECT * FROM produtos WHERE categoria IN ('papel  
aria', 'escritório', 'arte');  
SELECT * FROM clientes WHERE cidade NOT IN ('São  
Paulo', 'Rio de Janeiro');
```

8.4 BETWEEN

```
SELECT * FROM pedidos
```

```
WHERE criado_em BETWEEN '2026-01-01' AND '2026-06-30';  
-- Equivale a: criado_em >= '2026-01-01' AND criado_em <= '2026-06-30'
```

Atenção: Cuidado com BETWEEN em timestamps: o limite superior inclui o dia mas exclui as horas seguintes. Prefira criado_em >= '2026-01-01' AND criado_em < '2026-07-01'.

8.5 LIKE e ILIKE (case-insensitive)

```
SELECT * FROM clientes WHERE nome LIKE 'A%';  
-- começa com A  
SELECT * FROM clientes WHERE nome LIKE '%silva%';  
-- contém 'silva'  
SELECT * FROM clientes WHERE nome LIKE '_ndré';  
-- 1 caractere + 'ndré'  
SELECT * FROM clientes WHERE nome ILIKE 'ana%';  
-- ignora maiúscula/minúscula (Postgres)  
-- Escape do próprio %  
SELECT * FROM logs WHERE msg LIKE '%50\%%' ESCAPE  
'\';
```

8.6 Expressões regulares (Postgres)

```
SELECT * FROM clientes WHERE email ~ '^[a-z]+@empresa\.com$'; -- casa (case sensitive)  
SELECT * FROM clientes WHERE email ~* '@GMAIL\.com$'; -- case insensitive  
SELECT * FROM clientes WHERE cpf !~ '^[0-9]{11}$';  
-- não casa
```

8.7 IS NULL / IS NOT NULL / IS DISTINCT FROM

```
SELECT * FROM clientes WHERE cidade IS NULL;
SELECT * FROM clientes WHERE cidade IS NOT NULL;
-- Comparação segura contra NULL (retorna TRUE/FALSE, nunca NULL)
SELECT * FROM clientes WHERE cidade IS DISTINCT FROM 'São Paulo';
```

8.8 EXISTS e ANY/ALL

```
-- Clientes que já compraram
SELECT c.* FROM clientes c
  WHERE EXISTS (SELECT 1 FROM pedidos p WHERE p.cliente_id = c.id);
-- Produtos mais caros que TODAS as tarifas médias
SELECT * FROM produtos WHERE preco > ALL (SELECT preco_medio FROM tarifas);
-- Produtos mais caros que ALGUMA média
SELECT * FROM produtos WHERE preco > ANY (SELECT preco_medio FROM tarifas);
```

9. ORDER BY, LIMIT, OFFSET, FETCH

9.1 ORDER BY

```
SELECT * FROM produtos ORDER BY preco ASC;
    -- crescente (padrão)
SELECT * FROM produtos ORDER BY preco DESC;
    -- decrescente
SELECT * FROM produtos ORDER BY categoria, preco
DESC;
SELECT nome, preco * estoque AS valor
    FROM produtos ORDER BY valor DESC;
    -- pode ordenar por alias
SELECT * FROM produtos ORDER BY 3 DESC;
    -- posição da coluna (evite)
```

9.2 NULLs na ordenação

```
SELECT * FROM clientes ORDER BY cidade NULLS LAST
;    -- Postgres/Oracle
SELECT * FROM clientes ORDER BY cidade NULLS FIRST;
```

9.3 LIMIT / OFFSET

```
SELECT * FROM produtos ORDER BY id LIMIT 10;
    -- primeiros 10
SELECT * FROM produtos ORDER BY id LIMIT 10 OFFSET
20;    -- pula 20, pega 10
-- Padrão SQL:2008 (FETCH):
SELECT * FROM produtos ORDER BY id OFFSET 20 ROWS
    FETCH NEXT 10 ROWS ONLY;
-- SQL Server:
```

```
SELECT TOP 10 * FROM produtos ORDER BY id;
```

Atenção: OFFSET em valores grandes fica lento: o banco precisa varrer e descartar N linhas. Para paginação eficiente, veja o capítulo 21 (keyset pagination).

9.4 Paginação por página

```
-- Página P, tamanho N:  
SELECT * FROM produtos  
ORDER BY id  
LIMIT :N OFFSET (:P - 1) * :N;
```

9.5 TOP-N por grupo (prévia)

```
-- Os 3 produtos mais caros de CADA categoria  
SELECT * FROM (  
    SELECT p.*,  
           ROW_NUMBER() OVER (PARTITION BY categoria  
a ORDER BY preco DESC) AS rn  
    FROM produtos p  
) t WHERE rn <= 3;
```

10. Funções — string, numérica, data, condicional

10.1 Strings

```
SELECT length('lovable');           -- 7
SELECT upper('ana'), lower('ANA');  -- 'ANA'
, 'ana'
SELECT initcap('joão da silva');     -- 'João
Da Silva'
SELECT trim('  ola  ');              -- 'ola'
SELECT ltrim('---x', '-'), rtrim('x---', '-');
SELECT replace('a.b.c', '.', '-');  -- 'a-b-
c'
SELECT substring('lovable', 1, 4);   -- 'lova
'
SELECT position('v' in 'lovable');   -- 3
SELECT concat('ola', ' ', 'mundo');
SELECT format('id=%s nome=%s', 1, 'Ana');
SELECT split_part('a,b,c', ',', 2);  -- 'b'
SELECT lpad('42', 5, '0');           -- '0004
2'
SELECT reverse('abc');               -- 'cba'
```

10.2 Numéricas

```
SELECT abs(-7), sign(-3), mod(10,3);
SELECT round(3.14159, 2);           -- 3.14
SELECT ceil(2.1), floor(2.9);       -- 3 , 2
SELECT power(2, 10);                 -- 1024
SELECT sqrt(9);                      -- 3
SELECT random();                     -- 0..1
```

```
SELECT (random()*100)::int;
```

10.3 Data e hora

```
SELECT now(), current_date, current_time;
SELECT age(nascimento) FROM clientes;
-- intervalo
SELECT extract(year FROM criado_em) FROM pedidos
; -- 2026
SELECT date_trunc('month', criado_em) FROM pedidos;
-- primeiro dia do mês
SELECT criado_em + INTERVAL '7 days';
SELECT to_char(now(), 'DD/MM/YYYY HH24:MI');
SELECT to_date('17/07/2026', 'DD/MM/YYYY');
```

10.4 Condicionais

```
SELECT nome,
       CASE
           WHEN estoque = 0 THEN 'esgotado'
           WHEN estoque BETWEEN 1 AND 5 THEN 'baixo'
           ELSE 'ok'
       END AS status
FROM produtos;
SELECT COALESCE(apelido, nome) FROM clientes;
-- primeiro não-null
SELECT NULLIF(divisor, 0);
-- vira NULL se = 0
SELECT GREATEST(1, 5, 3), LEAST(1, 5, 3);
-- 5 , 1
```


10.5 Conversão

```
SELECT '42'::int + 8;  
SELECT to_number('1.234,56', 'FM999G999D00');    -  
- Postgres  
SELECT cast(preco AS text) FROM produtos;
```

11. Agregações — GROUP BY, HAVING, ROLLUP

11.1 Funções de agregação principais

```
SELECT count(*)          FROM pedidos;    -- total
de linhas

SELECT count(cidade)     FROM clientes;   -- ignora
NULLs

SELECT count(DISTINCT cliente_id) FROM pedidos;
SELECT sum(quantidade * preco_unit) FROM itens_pedido;
SELECT avg(preco), min(preco), max(preco) FROM produtos;
SELECT string_agg(nome, ', ' ORDER BY nome) FROM clientes;    -- concatena
SELECT array_agg(id ORDER BY criado_em) FROM pedidos WHERE cliente_id = 1;
SELECT jsonb_agg(row_to_json(p)) FROM produtos p;
```

11.2 GROUP BY

```
-- Vendas totais por categoria
SELECT p.categoria, sum(i.quantidade * i.preco_unit) AS total
FROM itens_pedido i
JOIN produtos p ON p.id = i.produto_id
GROUP BY p.categoria
ORDER BY total DESC;

-- Múltiplas colunas
SELECT p.categoria, date_trunc('month', ped.criado_em) AS mes,
```

```
        sum(i.quantidade * i.preco_unit) AS total
FROM itens_pedido i
JOIN produtos p    ON p.id = i.produto_id
JOIN pedidos ped   ON ped.id = i.pedido_id
GROUP BY p.categoria, mes
ORDER BY p.categoria, mes;
```

Atenção: Toda coluna do SELECT que não está dentro de uma função de agregação precisa aparecer no GROUP BY.

11.3 HAVING

```
-- Clientes com 5+ pedidos e ticket médio acima d
e 100
SELECT cliente_id,
       count(*)      AS qtd_pedidos,
       avg(total)    AS ticket_medio
FROM pedidos
GROUP BY cliente_id
HAVING count(*) >= 5 AND avg(total) > 100
ORDER BY ticket_medio DESC;
```

A diferença WHERE x HAVING é crucial: WHERE filtra linhas antes de agregar; HAVING filtra grupos depois de agregar.

11.4 GROUPING SETS, ROLLUP, CUBE

```
-- Vendas por categoria, por mês e o total geral,
numa única query
SELECT p.categoria,
```

```

        date_trunc('month', ped.criado_em) AS mes,
        sum(i.quantidade * i.preco_unit) AS total
FROM itens_pedido i
JOIN produtos p ON p.id = i.produto_id
JOIN pedidos ped ON ped.id = i.pedido_id
GROUP BY ROLLUP (p.categoria, mes);
-- CUBE: todas as combinações possíveis dos campos
SELECT p.categoria, ped.status, sum(i.quantidade)
FROM itens_pedido i
JOIN pedidos ped ON ped.id = i.pedido_id
JOIN produtos p ON p.id = i.produto_id
GROUP BY CUBE (p.categoria, ped.status);

```

11.5 FILTER (Postgres)

```

-- Contagens condicionais em uma passada só
SELECT count(*) AS total,
        count(*) FILTER (WHERE status = 'pago')
        AS pagos,
        count(*) FILTER (WHERE status = 'pendente'
) AS pendentes,
        sum(total) FILTER (WHERE status = 'pago')
        AS receita
FROM pedidos;

```

12. JOINS — INNER, LEFT, RIGHT, FULL, CROSS, SELF

Um JOIN combina linhas de duas ou mais tabelas usando uma condição. Todo modelo relacional sério depende de JOINS bem escritos.

12.1 INNER JOIN

```
-- Só clientes que têm pelo menos um pedido
SELECT c.id, c.nome, p.id AS pedido_id, p.total
FROM clientes c
INNER JOIN pedidos p ON p.cliente_id = c.id;
```

12.2 LEFT JOIN

```
-- TODOS os clientes; se não tiver pedido, colunas de pedido ficam NULL
SELECT c.id, c.nome, p.id AS pedido_id
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id;
-- Encontrar clientes SEM pedidos
SELECT c.*
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id
WHERE p.id IS NULL;
```

12.3 RIGHT JOIN e FULL OUTER JOIN

```
SELECT * FROM a RIGHT JOIN b ON a.id = b.a_id;  -  
- espelho de LEFT  
SELECT * FROM a FULL OUTER JOIN b ON a.id = b.a_id;  
-- traz linhas dos dois lados
```

12.4 CROSS JOIN

```
-- Produto cartesiano: cada linha de A combina com todas de B  
SELECT * FROM cores CROSS JOIN tamanhos;  
-- Muito útil para gerar séries  
SELECT d::date  
FROM generate_series('2026-01-01'::date, '2026-12-31'::date, INTERVAL '1 day') d;
```

12.5 SELF JOIN

```
-- Estrutura hierárquica: funcionário e seu gerente na mesma tabela  
SELECT f.nome AS funcionario, g.nome AS gerente  
FROM funcionarios f  
LEFT JOIN funcionarios g ON g.id = f.gerente_id  
;
```

12.6 USING e NATURAL JOIN

```
-- USING elimina o duplo de coluna quando os nomes batem  
SELECT id, nome FROM produtos JOIN produtos_extra USING (id);  
-- NATURAL JOIN casa por TODAS colunas de nome igual. Perigoso – evite.  
SELECT * FROM a NATURAL JOIN b;
```

12.7 Vários JOINS numa query

```
SELECT ped.id, c.nome, p.nome AS produto, i.quant  
idade, i.preco_unit  
FROM pedidos ped  
JOIN clientes c      ON c.id = ped.cliente_id  
JOIN itens_pedido i  ON i.pedido_id = ped.id  
JOIN produtos p      ON p.id = i.produto_id  
WHERE ped.criado_em >= now() - INTERVAL '30 days'  
  
ORDER BY ped.id;
```

12.8 LATERAL JOIN (Postgres)

```
-- Para cada cliente, pegar os 3 últimos pedidos  
SELECT c.id, c.nome, ult.id AS pedido_id, ult.cri  
ado_em  
FROM clientes c  
JOIN LATERAL (  
    SELECT id, criado_em  
    FROM pedidos p  
    WHERE p.cliente_id = c.id  
    ORDER BY p.criado_em DESC  
    LIMIT 3  
) ult ON TRUE;
```

12.9 Antipatterns

- Escrever vírgulas em vez de JOIN (FROM a, b WHERE a.id=b.a_id): funciona mas é ilegível e propenso a esquecer a condição,

gerando produto cartesiano.

- Fazer JOIN e depois esquecer o ON — quase sempre trava com milhões de linhas.
- Fazer LEFT JOIN e filtrar a tabela da direita no WHERE — vira INNER JOIN sem querer. Coloque o filtro no ON.

13. Subqueries e EXISTS

13.1 Subqueries escalares

```
-- Preço médio geral em cada linha (subquery esca  
lar retorna 1 valor)  
SELECT nome, preco,  
       (SELECT avg(preco) FROM produtos) AS preco  
_medio_geral  
FROM produtos;
```

13.2 Subqueries em FROM (derived table)

```
SELECT categoria, media  
FROM (SELECT categoria, avg(preco) AS media FRO  
M produtos GROUP BY categoria) t  
WHERE media > 50;
```

13.3 Subqueries em WHERE

```
SELECT * FROM produtos  
WHERE preco > (SELECT avg(preco) FROM produtos);  
SELECT * FROM clientes  
WHERE id IN (SELECT cliente_id FROM pedidos WHER  
E total > 1000);
```

13.4 EXISTS x IN — qual usar?

```
-- EXISTS costuma ser mais rápido quando a subque  
ry devolve muitas linhas  
SELECT c.*  
FROM clientes c  
WHERE EXISTS (SELECT 1 FROM pedidos p WHERE p.cl  
iente_id = c.id AND p.status='pago');
```

```
-- IN é ok para listas pequenas ou expressões simples
SELECT * FROM produtos WHERE categoria IN ('a','b','c');
```

13.5 Subqueries correlacionadas

```
-- Para cada pedido, quantos itens ele tem
SELECT p.id, p.total,
       (SELECT count(*) FROM itens_pedido i WHERE
        i.pedido_id = p.id) AS qtd_itens
FROM pedidos p;
```

Atenção: Subqueries correlacionadas rodam uma vez por linha — se possível, reescreva com JOIN + GROUP BY.

14. CTEs (WITH) e CTEs recursivas

14.1 CTE simples

Uma CTE (Common Table Expression) é uma consulta temporária, nomeada, definida no início da instrução. Torna queries complexas legíveis e reutilizáveis dentro da mesma execução.

```
WITH vendas_mes AS (  
    SELECT date_trunc('month', criado_em) AS mes,  
           sum(total) AS receita  
    FROM pedidos  
    WHERE status = 'pago'  
    GROUP BY 1  
)  
SELECT mes, receita,  
       receita - lag(receita) OVER (ORDER BY mes)  
AS variacao  
FROM vendas_mes  
ORDER BY mes;
```

14.2 Múltiplas CTEs encadeadas

```
WITH  
top_clientes AS (  
    SELECT cliente_id, sum(total) AS gasto  
    FROM pedidos GROUP BY cliente_id
```

```

        ORDER BY gasto DESC LIMIT 100
    ),
    detalhes AS (
        SELECT c.id, c.nome, c.cidade, tc.gasto
        FROM top_clientes tc
        JOIN clientes c ON c.id = tc.cliente_id
    )
    SELECT cidade, count(*) AS qtd, sum(gasto) AS total
    FROM detalhes
    GROUP BY cidade
    ORDER BY total DESC;

```

14.3 CTE recursiva

Ideal para hierarquias (organograma, categorias em árvore, grafos).

```

-- Categorias em árvore
CREATE TABLE categorias (
    id SERIAL PRIMARY KEY,
    nome TEXT NOT NULL,
    pai_id INT REFERENCES categorias(id)
);

-- Todos os descendentes de "Eletrônicos"
WITH RECURSIVE arvore AS (
    SELECT id, nome, pai_id, 1 AS nivel
    FROM categorias WHERE nome = 'Eletrônicos'
    UNION ALL
    SELECT c.id, c.nome, c.pai_id, a.nivel + 1

```

```

        FROM categorias c
        JOIN arvore a ON c.pai_id = a.id
    )
    SELECT repeat(' ', nivel - 1) || nome AS arvore
    FROM arvore ORDER BY nivel;

```

14.4 CTE modificadora (Postgres)

```

-- Move linhas antigas para um arquivo em uma única transação
WITH removidas AS (
    DELETE FROM pedidos WHERE criado_em < now() - INTERVAL '2 years'
    RETURNING *
)
INSERT INTO pedidos_arquivo SELECT * FROM removidas;

```

14.5 CTE MATERIALIZED / NOT MATERIALIZED

No Postgres 12+, você pode forçar (ou proibir) o cache da CTE:

```

WITH grandes AS NOT MATERIALIZED (
    SELECT * FROM produtos WHERE preco > 1000
)
SELECT * FROM grandes WHERE categoria = 'eletrônicos';
-- NOT MATERIALIZED permite o planejador "inlinar" a CTE – geralmente mais rápido.

```

15. Window Functions — OVER, PARTITION BY, ROW_NUMBER, LAG/LEAD

Window functions calculam valores por linha mas com base em um conjunto de linhas relacionadas — a 'janela'. Diferente de GROUP BY, elas não colapsam as linhas.

15.1 Sintaxe geral

```
funcao() OVER (  
    PARTITION BY colA          -- separa a janela  
em grupos  
    ORDER BY colB [DESC]      -- ordena dentro do  
grupo  
    ROWS BETWEEN ... AND ... -- opcional: define  
o "quadro" (frame)  
)
```

15.2 ROW_NUMBER, RANK, DENSE_RANK

```
SELECT id, categoria, preco,  
       ROW_NUMBER() OVER (PARTITION BY categoria  
ORDER BY preco DESC) AS rn,  
       RANK()          OVER (PARTITION BY categoria  
ORDER BY preco DESC) AS rk,  
       DENSE_RANK() OVER (PARTITION BY categoria  
ORDER BY preco DESC) AS drk  
FROM produtos;
```

15.3 LAG / LEAD — comparar com a linha anterior/próxima

```
SELECT criado_em::date AS dia,
       receita,
       LAG(receita) OVER (ORDER BY criado_em) AS
ontem,
       receita - LAG(receita) OVER (ORDER BY cria
do_em) AS variacao
FROM receitas_diarias;
```

15.4 Somas móveis / cumulativas

```
SELECT dia, receita,
       sum(receita) OVER (ORDER BY dia) AS acumul
ada,
       avg(receita) OVER (ORDER BY dia ROWS BETWE
EN 6 PRECEDING AND CURRENT ROW) AS media_7d
FROM receitas_diarias;
```

15.5 NTILE — dividir em quartis, decis

```
SELECT id, gasto,
       NTILE(4) OVER (ORDER BY gasto DESC) AS qua
rtil
FROM clientes;
```

15.6 First_value / Last_value

```
SELECT cliente_id, criado_em, total,
       FIRST_VALUE(total) OVER w AS primeiro_pedi
do,
       LAST_VALUE(total) OVER w AS ultimo_pedido
FROM pedidos
```

```
WINDOW w AS (PARTITION BY cliente_id ORDER BY c  
riado_em  
ROWS BETWEEN UNBOUNDED PRECEDING A  
ND UNBOUNDED FOLLOWING);
```

15.7 Top-N por grupo (padrão clássico)

```
-- 3 produtos mais caros por categoria  
SELECT * FROM (  
    SELECT p.*,  
           ROW_NUMBER() OVER (PARTITION BY categori  
a ORDER BY preco DESC) AS rn  
    FROM produtos p  
) t WHERE rn <= 3;
```


16. Views e Materialized Views

16.1 View

Uma view é uma consulta salva com nome. Toda vez que você lê a view, o SGBD executa a query por trás dela. Ótimo para encapsular regras e simplificar consultas para relatórios.

```
CREATE VIEW v_vendas_categoria AS
SELECT p.categoria,
       date_trunc('month', ped.criado_em) AS mes,
       sum(i.quantidade * i.preco_unit) AS total
FROM itens_pedido i
JOIN produtos p ON p.id = i.produto_id
JOIN pedidos ped ON ped.id = i.pedido_id
GROUP BY 1,2;

SELECT * FROM v_vendas_categoria WHERE categoria
= 'papelaria';
```

16.2 Views atualizáveis

```
-- Views simples (sem GROUP BY, JOIN, DISTINCT, e
tc.) podem ser alvo de INSERT/UPDATE/DELETE.
-- Para casos complexos, use INSTEAD OF triggers.
```

16.3 Materialized View

Guarda o resultado da query em disco. Consultas são rápidas, mas os dados

envelhecem. É preciso atualizar manualmente (ou por cron).

```
CREATE MATERIALIZED VIEW mv_kpi_diario AS
SELECT date_trunc('day', criado_em) AS dia,
       count(*) AS pedidos,
       sum(total) AS receita
FROM pedidos WHERE status = 'pago'
GROUP BY 1;

-- Índice numa MV é permitido e muito recomendado
:
CREATE INDEX idx_mv_kpi_dia ON mv_kpi_diario(dia)
;

-- Recomputar (bloqueando escritas):
REFRESH MATERIALIZED VIEW mv_kpi_diario;

-- Recomputar sem bloquear leituras (precisa de índice UNIQUE):
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_kpi_diario;
```

17. Índices — B-Tree, Hash, GIN, GiST, parciais e compostos

Um índice é uma estrutura auxiliar que acelera buscas — em troca de escrita mais lenta e mais espaço em disco. Índices são a diferença entre uma query que roda em 1 ms e a mesma query rodando em 30 segundos.

17.1 B-Tree — o padrão

```
CREATE INDEX idx_pedidos_cliente ON pedidos(cliente_id);  
CREATE INDEX idx_pedidos_criado_desc ON pedidos(criado_em DESC);  
-- Composto (a ordem das colunas importa!)  
CREATE INDEX idx_pedidos_cliente_criado  
ON pedidos(cliente_id, criado_em DESC);
```

17.2 UNIQUE index

```
CREATE UNIQUE INDEX uk_clientes_email ON clientes  
(lower(email));
```

17.3 Índice parcial

```
-- Só indexa as linhas que interessam — menor e mais rápido  
CREATE INDEX idx_pedidos_abertos  
ON pedidos(criado_em)
```

```
WHERE status = 'aberto';
```

17.4 Índice por expressão

```
CREATE INDEX idx_lower_email ON clientes(lower(email));
```

-- Query precisa usar a MESMA expressão para o índice ser aproveitado:

```
SELECT * FROM clientes WHERE lower(email) = 'ana@x.com';
```

17.5 Hash, GIN, GiST, BRIN (Postgres)

-- HASH: igualdade apenas

```
CREATE INDEX idx_h_status ON pedidos USING HASH (status);
```

-- GIN: perfeito para JSONB, arrays, tsvector

```
CREATE INDEX idx_gin_tags ON produtos USING GIN (tags); -- tags TEXT[]
```

```
CREATE INDEX idx_gin_meta ON produtos USING GIN (metadata jsonb_path_ops);
```

-- GiST: dados geométricos, ranges

```
CREATE INDEX idx_gist_periodo ON reservas USING GIST (periodo);
```

-- BRIN: tabelas GIGANTES ordenadas naturalmente por data

```
CREATE INDEX idx_brin_criado ON logs USING BRIN (criado_em);
```

17.6 Quando o índice NÃO é usado

- Uso de função na coluna sem índice funcional: `WHERE lower(email)=....`
- Comparação de tipos diferentes que força CAST implícito.
- LIKE com % no início (LIKE '%foo'). Use índice trigram (pg_trgm).
- Query traz muitas linhas — o planner escolhe seq scan porque é mais rápido.
- Tabela muito pequena — seq scan vence.

17.7 EXPLAIN — ver se o índice está sendo usado

```
EXPLAIN ANALYZE
SELECT * FROM pedidos WHERE cliente_id = 42 AND c
riado_em > now() - INTERVAL '30 days';
```

18. Transações, ACID e níveis de isolamento

Uma transação agrupa comandos em uma unidade atômica: ou tudo é aplicado, ou nada é.

18.1 BEGIN, COMMIT, ROLLBACK

```
BEGIN;  
    UPDATE contas SET saldo = saldo - 100 WHERE id  
= 1;  
    UPDATE contas SET saldo = saldo + 100 WHERE id  
= 2;  
COMMIT;  
  
-- se algo der errado:  
ROLLBACK;
```

18.2 As propriedades ACID

- Atomicidade: a transação é indivisível.
- Consistência: leva o banco de um estado válido para outro (respeitando constraints).
- Isolamento: transações concorrentes não se enxergam parcialmente.
- Durabilidade: uma vez COMMITada, sobrevive a quedas.

18.3 Savepoints

```
BEGIN;
  INSERT INTO logs (msg) VALUES ('inicio');
  SAVEPOINT sp1;
  UPDATE contas SET saldo = saldo - 100 WHERE id = 1;
  -- imagine que dá erro
  ROLLBACK TO SAVEPOINT sp1;
  -- desfaz só até o savepoint
  INSERT INTO logs (msg) VALUES ('recuperado');
COMMIT;
```

18.4 Níveis de isolamento (SQL padrão)

Nível	Dirty read	Non-repeatable read	Phantom read
READ UNCOMMITTED	sim*	sim	sim
READ COMMITTED (padrão Postgres)	não	sim	sim
REPEATABLE READ	não	não	sim*
SERIALIZABLE	não	não	não

```
-- Ajustar isolamento
BEGIN ISOLATION LEVEL SERIALIZABLE;
  -- ... operações ...
COMMIT;

-- Postgres: no REPEATABLE READ os phantoms não o
correm (implementação MVCC snapshot).
```

18.5 Locks explícitos

```
BEGIN;
SELECT * FROM contas WHERE id = 1 FOR UPDATE;
-- trava a linha
UPDATE contas SET saldo = saldo - 100 WHERE id = 1;
COMMIT;
```

```
-- Trava toda a tabela (raro)  
LOCK TABLE contas IN SHARE MODE;
```

18.6 Deadlocks

Ocorrem quando duas transações esperam recursos travados uma pela outra. O SGBD detecta e aborta uma delas — sua aplicação precisa fazer retry. Padrão prático: sempre acessar recursos na mesma ordem.

19. Procedures, Functions e Triggers

19.1 Function em PL/pgSQL

```
CREATE OR REPLACE FUNCTION total_pedido(p_id BIGINT)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
DECLARE
    v_total NUMERIC := 0;
BEGIN
    SELECT COALESCE(sum(quantidade * preco_unit), 0)
    INTO v_total
    FROM itens_pedido
    WHERE pedido_id = p_id;
    RETURN v_total;
END;
$$;
-- Usar
SELECT total_pedido(1);
```

19.2 Procedure (Postgres 11+)

```
CREATE OR REPLACE PROCEDURE arquivar_pedidos(dias
INT)
LANGUAGE plpgsql
AS $$
BEGIN
    WITH removidos AS (
```

```

        DELETE FROM pedidos WHERE criado_em < now() -
        (dias || ' days')::interval
        RETURNING *
    )
    INSERT INTO pedidos_arquivo SELECT * FROM remov
idos;
    COMMIT;    -- procedures podem gerenciar transaç
ões
END;
$$;
CALL arquivar_pedidos(365);

```

19.3 Triggers

Um trigger executa código automaticamente antes/depois de INSERT/UPDATE/DELETE.

```

-- 1) Função que será chamada
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    NEW.updated_at := now();
    RETURN NEW;
END;
$$;

-- 2) Trigger que a conecta
CREATE TRIGGER trg_produtos_updated
BEFORE UPDATE ON produtos
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

19.4 Trigger para auditoria

```
CREATE TABLE auditoria (  
    id BIGSERIAL PRIMARY KEY,  
    tabela TEXT, operacao TEXT, id_afetado BIGINT,  
    dados_old JSONB, dados_new JSONB, quando TIMEST  
AMPTZ DEFAULT now()  
);  
  
CREATE OR REPLACE FUNCTION log_auditoria()  
RETURNS TRIGGER LANGUAGE plpgsql AS $$  
BEGIN  
    INSERT INTO auditoria (tabela, operacao, id_afe  
tado, dados_old, dados_new)  
    VALUES (TG_TABLE_NAME, TG_OP,  
            COALESCE(NEW.id, OLD.id),  
            CASE WHEN TG_OP <> 'INSERT' THEN row_to  
_json(OLD)::jsonb END,  
            CASE WHEN TG_OP <> 'DELETE' THEN row_to  
_json(NEW)::jsonb END);  
    RETURN COALESCE(NEW, OLD);  
END;  
$$;  
  
CREATE TRIGGER trg_audit_produtos  
AFTER INSERT OR UPDATE OR DELETE ON produtos  
FOR EACH ROW EXECUTE FUNCTION log_auditoria();
```

Atenção: Triggers rodam para toda operação — abusar deles pode transformar cargas simples em pesadelos de performance. Prefira lógica na aplicação e reserve triggers para invariantes que precisam ser garantidos pelo banco.

20. Normalização — 1FN, 2FN, 3FN, BCNF

Normalização é o processo de organizar dados para eliminar redundância e evitar anomalias de atualização, inserção e remoção.

20.1 Primeira Forma Normal (1FN)

Toda coluna guarda um valor atômico; sem listas dentro de uma célula.

```
-- Errado (viola 1FN)
| id | cliente | telefones |
|----|-----|-----|
| 1  | Ana     | '11 999, 11 888' |

-- Certo
| id | cliente_id | telefone |
|----|-----|-----|
| 1  | 1          | 11 99999... |
| 2  | 1          | 11 88888... |
```

20.2 Segunda Forma Normal (2FN)

Está em 1FN e nenhuma coluna não-chave depende apenas de parte da chave composta.

```
-- Chave: (pedido_id, produto_id).
-- 'nome_produto' depende só de produto_id -> mover para tabela produtos.
```

20.3 Terceira Forma Normal (3FN)

Está em 2FN e nenhuma coluna depende de outra coluna que não seja chave.

```
-- Errado: cep, cidade e estado em clientes, redundantes.  
-- Certo: tabela ceps(cep PK, cidade, estado), clientes referencia cep.
```

20.4 Forma Normal de Boyce-Codd (BCNF)

Toda dependência funcional tem uma superchave à esquerda. Mais estrita que a 3FN, cobre casos raros mas patológicos.

20.5 Desnormalizar? Quando faz sentido

- Cache de agregações em Materialized Views ou tabelas de fatos.
- Colunas denormalizadas para busca full-text.
- Datawarehouse (esquemas estrela e floco) — analytics não sofre com escritas concorrentes.

21. Ler o banco em blocos: paginação, keyset e cursores

Ler tudo de uma tabela grande de uma vez pode estourar memória, travar a rede e quebrar o timeout da aplicação. Em vez disso, lemos um bloco por vez e processamos, guardando um marcador (bookmark) para continuar de onde paramos na próxima chamada.

Este capítulo mostra três técnicas — do jeito ingênuo ao jeito profissional.

21.1 O jeito ingênuo: LIMIT + OFFSET

```
SELECT id, nome, criado_em
  FROM clientes
 ORDER BY id
 LIMIT 1000 OFFSET 0;      -- primeira página

SELECT id, nome, criado_em
  FROM clientes
 ORDER BY id
 LIMIT 1000 OFFSET 1000;  -- segunda página

SELECT id, nome, criado_em
  FROM clientes
 ORDER BY id
```

```
LIMIT 1000 OFFSET 2000;    -- terceira página
```

Atenção: Cada página com OFFSET N faz o banco varrer e descartar N linhas antes de devolver a página. OFFSET 1.000.000 é dolorosamente lento — mesmo com índice — porque o índice ainda precisa percorrer 1 milhão de entradas.

Além disso, se linhas forem inseridas ou removidas entre as chamadas, a paginação por OFFSET pode pular ou repetir registros — a posição N mudou de significado.

21.2 O jeito certo: paginação por chave (keyset / seek pagination)

A ideia central: em vez de dizer “me dê a página X”, dizemos “me dê os próximos N registros depois de um marcador que eu já conheço”. O marcador é a última chave ordenada da página anterior. O banco usa o índice para saltar direto para essa posição.

```
-- Primeira leitura (sem marcador)
SELECT id, nome, criado_em
  FROM clientes
 ORDER BY id
  LIMIT 1000;

-- Guarda o último id retornado, ex.: 1000
```

```

-- Próxima leitura, continuando de onde parou:
SELECT id, nome, criado_em
  FROM clientes
 WHERE id > 1000                                -- << o "cursor lógico"
 ORDER BY id
 LIMIT 1000;

-- E assim por diante:
SELECT ... FROM clientes WHERE id > 2000 ORDER BY
id LIMIT 1000;

```

Como o banco usa o índice de id para pular direto para id > 1000, a página 1000 custa o mesmo que a página 1. Não há degradação com o tempo.

21.3 Keyset com ordenação composta

Se você ordena por várias colunas (ex.: criado_em DESC, id DESC), o marcador precisa ser uma tupla. A comparação usa a forma canônica de tuplas do SQL:

```

-- Ordenação: mais recentes primeiro; desempate p
or id
-- Primeira página:
SELECT id, cliente_id, criado_em, total
  FROM pedidos

```



```

ORDER BY criado_em DESC, id DESC
LIMIT 100;

-- Suponha que o último item veio com:
--   ultimo_criado_em = '2026-07-17 14:33:21+00'
--   ultimo_id       = 987654
-- Página seguinte (compara TUPLAS):
SELECT id, cliente_id, criado_em, total
  FROM pedidos
 WHERE (criado_em, id) < ('2026-07-17 14:33:21+00'
', 987654)
 ORDER BY criado_em DESC, id DESC
LIMIT 100;

```

Nota: A comparação de tuplas $(a,b) < (x,y)$ equivale a $a < x$ OR $(a = x \text{ AND } b < y)$. Isso trata os empates corretamente, coisa que `WHERE criado_em < X` sozinho não faz.

21.4 O que guardar em memória (o 'estado' entre lotes)

- Basta o último valor das colunas de ordenação (o marcador). Bytes, não megabytes.
- Nada de guardar a lista inteira de IDs já lidos — o marcador substitui isso.
- Se o processo pode reiniciar, salve o marcador em disco (arquivo, Redis, tabela de progresso).

```
-- Tabela para persistir o progresso do processamento em batch
CREATE TABLE batch_progress (
    job_name      TEXT PRIMARY KEY,
    last_id       BIGINT,
    last_ts       TIMESTAMPTZ,
    atualizado    TIMESTAMPTZ DEFAULT now()
);

-- Ao terminar cada bloco:
INSERT INTO batch_progress (job_name, last_id, last_ts)
VALUES ('export_clientes', 1000, now())
ON CONFLICT (job_name)
DO UPDATE SET last_id = EXCLUDED.last_id, last_ts
= EXCLUDED.last_ts, atualizado = now();
```

21.5 Implementação em Python (psycopg) — carrega, usa, continua

```
# pip install psycopg[binary]
import psycopg
DSN = "postgresql://user:pass@localhost/loja"
CHUNK = 1000
def carregar_bloco(conn, last_id):
    with conn.cursor() as cur:
        cur.execute(
            """
            SELECT id, nome, email
            FROM clientes
            WHERE id > %s                -- continua d
e onde parou
```

```

        ORDER BY id
        LIMIT %s
        """
        (last_id, CHUNK),
    )
    return cur.fetchall() # devolve a lista de tuplas para a memória
def processar(rows):
    # aqui você usa os dados carregados: envia e-mail, transforma, exporta CSV...
    for row in rows:
        print(row)
def salvar_progresso(conn, last_id):
    with conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO batch_progress (job_name,
            last_id) VALUES ('export_clientes', %s)
            ON CONFLICT (job_name) DO UPDATE SET
            last_id = EXCLUDED.last_id, atualizado = now()
            """,
            (last_id,),
        )
        conn.commit()
def ler_progresso(conn):
    with conn.cursor() as cur:
        cur.execute("SELECT last_id FROM batch_progress WHERE job_name = 'export_clientes'")
        r = cur.fetchone()
        return r[0] if r else 0

```

```

with psycopg.connect(DSN) as conn:
    last_id = ler_progresso(conn)      # retoma do
    ponto salvo (ou 0 na 1ª vez)
    while True:
        bloco = carregar_bloco(conn, last_id)  #
        1) lê parte do banco para a memória
        if not bloco:
            break
    # 2) acabou
        processar(bloco)
    # 3) usa os dados
        last_id = bloco[-1][0]
    # 4) atualiza marcador (id do último)
        salvar_progresso(conn, last_id)
    # 5) grava para poder retomar depois
        print("Concluído em", last_id)

```

21.6 Implementação em Node.js (pg) — versão async iterator

```

// npm i pg
import pg from 'pg';
const { Pool } = pg;
const pool = new Pool({ connectionString: process.env.DATABASE_URL });
const CHUNK = 1000;
async function* lerEmBlocos(tabela, cols, orderCol='id') {
    let cursor = 0;
    while (true) {
        const { rows } = await pool.query(

```

```

        `SELECT ${cols} FROM ${tabela}
          WHERE ${orderCol} > $1
          ORDER BY ${orderCol}
          LIMIT $2`,
        [cursor, CHUNK]
    );
    if (rows.length === 0) return;
    yield rows; // devolve
o bloco atual para quem consome
    cursor = rows[rows.length - 1][orderCol]; //
avança o marcador
}
}
// Uso:
for await (const bloco of lerEmBlocos('clientes',
'id, nome, email')) {
    console.log('bloco de', bloco.length, 'linhas')
;
    // ...processa...
}

```

21.7 Filtros adicionais + keyset

O keyset combina normalmente com WHERE — só cuide de manter o índice compatível com a cláusula. O padrão vencedor é um índice composto que começa pelos filtros de igualdade e termina pelas colunas do ORDER

BY:

```
-- Suporte de índice para: WHERE cliente_id = ? A
ND (criado_em, id) < (?, ?)
CREATE INDEX idx_pedidos_cli_criado_id
  ON pedidos (cliente_id, criado_em DESC, id DESC
);
-- Consulta paginada por keyset:
SELECT id, criado_em, total
  FROM pedidos
 WHERE cliente_id = 42
   AND (criado_em, id) < ($1, $2)
 ORDER BY criado_em DESC, id DESC
 LIMIT 100;
```

21.8 Ida e volta (voltar uma página)

Keyset é direcional. Para permitir 'página anterior', mantenha uma pilha de marcadores visitados no cliente e faça a consulta inversa quando necessário (troca > por < e o ORDER BY por seu oposto).

21.9 Cursores server-side — quando o banco lembra por você

O SQL padrão tem cursores. O servidor mantém a posição da leitura enquanto você chama FETCH. Não precisa de marcador

manual — mas exige uma transação aberta.

```
BEGIN;
DECLARE cli_cursor CURSOR FOR
    SELECT id, nome, email FROM clientes ORDER BY id;
FETCH 1000 FROM cli_cursor;      -- 1º bloco
-- ... processa em memória ...
FETCH 1000 FROM cli_cursor;      -- 2º bloco
FETCH 1000 FROM cli_cursor;      -- 3º bloco
-- ...
CLOSE cli_cursor;
COMMIT;
```

Atenção: Cursores mantêm um snapshot da transação. Se o job dura horas, você segura um snapshot longo no Postgres e atrapalha o VACUUM. Para jobs longos e reinicializáveis, prefira keyset.

21.10 Cursor com HOLD (sobrevive ao COMMIT — Postgres)

```
BEGIN;
DECLARE big_cursor CURSOR WITH HOLD FOR
    SELECT * FROM eventos WHERE dia = current_date;
COMMIT;      -- o cursor CONTINUA existindo fora da
              transação
FETCH 500 FROM big_cursor;
FETCH 500 FROM big_cursor;
CLOSE big_cursor;
```

21.11 Escolha rápida

Cenário	Escolha
Paginação em interface web ('próxima página')	Keyset
Ir para página arbitrária ('página 587')	OFFSET (com cautela; só se a tabela é pequena)
Job em background que pode reiniciar	Keyset + tabela batch_progress
Exportar tudo em um único processo, transação curta	Cursor server-side
Streaming linha a linha para outro sistema	Cursor + FETCH ou keyset em lotes

21.12 Erros comuns em paginação

- Ordenar por uma coluna não única sem desempate: registros no limite entre páginas somem/duplicam.
- Trocar de coluna de ordenação sem trocar o marcador: as tuplas ficam inconsistentes.
- Comparar as colunas do marcador com AND em vez de com tupla: perde linhas do empate.
- Confiar em OFFSET quando o dataset é grande — vira gargalo silencioso.
- Segurar cursor por horas — atrapalha o VACUUM/autovacuum.

22. Cursores server-side e streaming em Python/Node

22.1 psycopg — server-side cursor por nome

Um cursor nomeado em psycopg é streaming de verdade: o servidor manda linhas em lotes e a biblioteca faz FETCH sob demanda enquanto você itera.

```
import psycopg
with psycopg.connect(DSN) as conn:
    with conn.cursor(name="stream_clientes") as cur:
        # nome => cursor server-side
        cur.itersize = 5000
        # tamanho do lote interno
        cur.execute("SELECT id, nome, email FROM
clientes ORDER BY id")
        for row in cur:
            # for streamado, sem carregar tudo
            processar(row)
```

22.2 node-postgres — pg-cursor

```
// npm i pg pg-cursor
import pg from 'pg';
import Cursor from 'pg-cursor';
const client = new pg.Client({ connectionString:
process.env.DATABASE_URL });
await client.connect();
```

```

const cursor = client.query(new Cursor('SELECT *
FROM eventos WHERE dia=$1', [new Date()]));
async function readAll() {
  while (true) {
    const rows = await new Promise((res, rej) =>
      cursor.read(500, (err, rows) => err ? rej(e
rr) : res(rows)));
    if (rows.length === 0) break;
    for (const r of rows) await handle(r);
  }
  await cursor.close();
  await client.end();
}
readAll();

```

22.3 JDBC (Java) — fetchSize + autoCommit=false

```

conn.setAutoCommit(false);
// pré-requisito no Postgres
try (PreparedStatement ps = conn.prepareStatement
("SELECT id, payload FROM eventos")) {
  ps.setFetchSize(1000);
  // sem isso o driver carrega TUDO
  try (ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
      processar(rs.getLong(1), rs.getString
(2));
    }
  }
}

```

22.4 Copiando para um arquivo (streaming pesado)

-- Baixar milhões de linhas em CSV sem passar por memória do cliente:

```
COPY (SELECT * FROM eventos WHERE dia = current_date) TO STDOUT WITH CSV HEADER;
```

Do lado da shell:

```
psql -c "COPY (SELECT * FROM eventos) TO STDOUT WITH CSV HEADER" > eventos.csv
```

23. Performance — EXPLAIN, planos de execução, estatísticas

23.1 EXPLAIN e EXPLAIN ANALYZE

```
EXPLAIN SELECT * FROM pedidos WHERE cliente_id = 42;
```

```
-- Apenas o plano estimado.
```

```
EXPLAIN (ANALYZE, BUFFERS, VERBOSE)
```

```
SELECT * FROM pedidos WHERE cliente_id = 42;
```

```
-- ANALYZE executa a query e mede tempos reais.
```

23.2 Como ler o plano

- Seq Scan: leitura sequencial. Ok em tabelas pequenas; ruim em grandes.
- Index Scan / Index Only Scan: usa índice — geralmente o que você quer.
- Bitmap Heap Scan: combina vários índices.
- Nested Loop: bom quando o lado externo é pequeno.
- Hash Join: bom para grandes datasets com igualdade.
- Merge Join: bom quando ambos lados já vêm ordenados.

- Sort caro? Adicione índice na coluna de ORDER BY/GROUP BY.

23.3 Estatísticas

```
ANALYZE clientes;           -- atualiza estatísticas
                             da tabela
VACUUM ANALYZE clientes; -- limpa mortos + atuali
                             za estatísticas
-- Postgres roda AUTOVACUUM em background, mas em
cargas pesadas ajuste os parâmetros.
```

23.4 Ajustes práticos que rendem muito

- Criar índice apropriado (composto na ordem igualdades → range → order by).
- Substituir SELECT * por só as colunas necessárias.
- Evitar funções na coluna filtrada (WHERE date_trunc('day', criado_em)=...).
- Usar EXISTS em vez de IN (SELECT ...) quando a subquery é grande.
- Materializar agregados frequentes em materialized views.

- Particionar tabelas gigantes por data (declarative partitioning).

24. Segurança — SQL Injection, prepared statements, roles

24.1 SQL Injection

A vulnerabilidade #1 em aplicações web ainda hoje. Ocorre quando dados do usuário são concatenados diretamente em uma string SQL.

```
# ERRADO
```

```
q = f"SELECT * FROM users WHERE email = '{email}'"
```

```
# Se email = "x' OR '1'='1", o SQL vira:
```

```
# SELECT * FROM users WHERE email = 'x' OR '1'='1'
# -> retorna tudo!
```

```
# CERTO – prepared statements com parâmetros
```

```
cur.execute("SELECT * FROM users WHERE email = %s", (email,))
```

24.2 Roles e privilégios (Postgres)

```
CREATE ROLE app_ro LOGIN PASSWORD 'segredo';
GRANT USAGE ON SCHEMA public TO app_ro;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_ro;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT ON TABLES TO app_ro;
CREATE ROLE app_rw LOGIN PASSWORD 'outra';
```

```
GRANT USAGE ON SCHEMA public TO app_rw;  
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES  
IN SCHEMA public TO app_rw;
```

24.3 Row-Level Security (RLS)

```
ALTER TABLE pedidos ENABLE ROW LEVEL SECURITY;  
CREATE POLICY pedidos_donos ON pedidos  
  FOR SELECT USING (cliente_id = current_setting(  
    'app.user_id')::bigint);  
-- A aplicação faz antes de cada request:  
SET LOCAL app.user_id = '42';
```

24.4 Boas práticas

- Nunca conecte a aplicação como superuser.
- Trate segredos como segredos: fora do código, em cofres/variáveis de ambiente.
- Sempre criptografe conexões (SSL/TLS).
- Auditoria: registre quem fez o quê (triggers, logs, extensões pg_audit).
- Backups criptografados e testados regularmente.

25. JSON e dados semiestruturados no SQL moderno

Bancos relacionais modernos guardam e consultam JSON sem esforço. É a ponte entre o modelo relacional rígido e o mundo flexível dos documentos.

25.1 Colunas JSONB

```
CREATE TABLE eventos (  
    id BIGSERIAL PRIMARY KEY,  
    tipo TEXT,  
    dados JSONB NOT NULL  
);  
  
INSERT INTO eventos (tipo, dados) VALUES  
    ('login',    '{"user_id":42, "ip":"1.2.3.4"}'),  
    ('compra',   '{"user_id":42, "produto_id":9, "val  
or":199.90}');
```

25.2 Operadores

```
SELECT dados->'user_id'          FROM eventos;  
-- JSONB  
  
SELECT dados->>'user_id'         FROM eventos;  
-- texto  
  
SELECT (dados->>'valor')::numeric FROM eventos WH  
ERE tipo='compra';  
  
SELECT * FROM eventos WHERE dados @> '{"user_id":  
42}';    -- contém
```

```
SELECT * FROM eventos WHERE dados ? 'ip';
      -- tem a chave 'ip'
SELECT dados #> '{endereco,cidade}';
      -- caminho
```

25.3 Índice GIN

```
CREATE INDEX idx_eventos_dados ON eventos USING G
IN (dados jsonb_path_ops);
-- acelera consultas com @>, ? e ?|
```

25.4 jsonb_path_query e jsonb_set

```
SELECT jsonb_path_query(dados, '$.itens[*].preco'
) FROM pedidos_json;
UPDATE eventos
      SET dados = jsonb_set(dados, '{ip}', '"9.9.9.9
"', true)
WHERE id = 1;
```

25.5 Quando usar JSONB (e quando não)

- Use JSONB quando o formato varia por linha e você raramente filtra por campos específicos.
- Use colunas relacionais quando o campo é consultado o tempo todo (mais rápido, mais tipado, mais fácil de indexar de forma composta).

- Padrão híbrido: colunas 'quentes' relacionais + um JSONB para o resto.

26. Backup, restore, replicação e migrações

26.1 pg_dump / pg_restore

```
# Backup lógico da base inteira
pg_dump -Fc -h localhost -U postgres loja > loja.dump

# Restaurar
pg_restore -h localhost -U postgres -d loja_nova loja.dump

# Só schema, só dados
pg_dump --schema-only loja > schema.sql
pg_dump --data-only    loja > dados.sql
```

26.2 Backup físico e PITR

Base backup + WAL archiving permitem Point-In-Time Recovery: restaurar o banco em qualquer instante do passado. Essencial em produção séria.

26.3 Replicação

- Streaming replication: réplica quente, quase em tempo real. Ideal para leitura escalada e failover.
- Logical replication (Postgres 10+): publica tabelas específicas, útil para ETL contínuo.

- Cascading: réplica alimenta outras réplicas.

26.4 Migrações versionadas

Use uma ferramenta de migrações no seu projeto — Flyway, Liquibase, golang-migrate, Alembic, Prisma, Drizzle, Knex. Cada mudança de schema vira um arquivo versionado, executado em ordem em todos os ambientes.

```
-- Exemplo de convenção Flyway/golang-migrate  
V001__create_clientes.sql  
V002__add_email_index.sql  
V003__add_column_telefone.sql
```

27. Exercícios resolvidos e projetos práticos

Cada exercício tem enunciado e uma solução comentada. Tente resolver antes de olhar.

27.1 Consultas básicas

1. Liste o nome e o e-mail dos clientes ativos, em ordem alfabética.

```
SELECT nome, email
  FROM clientes
 WHERE ativo = TRUE
 ORDER BY nome;
```

2. Quantos pedidos foram feitos por mês em 2026?

```
SELECT date_trunc('month', criado_em) AS mes,
       count(*) AS pedidos
  FROM pedidos
 WHERE criado_em >= '2026-01-01' AND criado_em <
 '2027-01-01'
 GROUP BY mes
 ORDER BY mes;
```

3. Ticket médio por cliente (só clientes com pelo menos 1 pedido).

```
SELECT c.id, c.nome, avg(p.total) AS ticket_medio
FROM clientes c
JOIN pedidos p ON p.cliente_id = c.id
GROUP BY c.id
ORDER BY ticket_medio DESC;
```

27.2 JOINS

4. Produtos que nunca foram vendidos.

```
SELECT p.*
FROM produtos p
LEFT JOIN itens_pedido i ON i.produto_id = p.id
WHERE i.produto_id IS NULL;
```

5. Top 5 produtos por receita total.

```
SELECT p.id, p.nome, sum(i.quantidade * i.preco_u
nit) AS receita
FROM itens_pedido i
JOIN produtos p ON p.id = i.produto_id
GROUP BY p.id, p.nome
ORDER BY receita DESC
LIMIT 5;
```

27.3 Window functions

6. Para cada cliente, mostre seus pedidos numerados do mais recente para o mais antigo.

```
SELECT cliente_id, id AS pedido_id, criado_em,  
       ROW_NUMBER() OVER (PARTITION BY cliente_id  
ORDER BY criado_em DESC) AS n  
FROM pedidos;
```

7. Variação percentual da receita em relação ao mês anterior.

```
WITH mensal AS (  
    SELECT date_trunc('month', criado_em) AS mes, s  
um(total) AS r  
    FROM pedidos GROUP BY 1  
)  
SELECT mes, r,  
       LAG(r) OVER (ORDER BY mes) AS r_anterior,  
       round(100.0 * (r - LAG(r) OVER (ORDER BY m  
es)) / NULLIF(LAG(r) OVER (ORDER BY mes),0), 2)  
       AS variacao_pct  
FROM mensal  
ORDER BY mes;
```

27.4 Paginação em blocos (revisando o capítulo 21)

8. Escreva a query da 2ª leitura, sabendo que o último id da 1ª foi 1500 e cada bloco tem 500.

```
SELECT id, nome, email  
FROM clientes
```



```
WHERE id > 1500
ORDER BY id
LIMIT 500;
```

9. Idem, ordenando por (criado_em DESC, id DESC), sendo o último marcador ('2026-07-17 10:00', 987).

```
SELECT id, nome, criado_em
FROM clientes
WHERE (criado_em, id) < ('2026-07-17 10:00', 987
)
ORDER BY criado_em DESC, id DESC
LIMIT 500;
```

27.5 Projetos práticos sugeridos

- Modele um sistema de biblioteca (livros, autores, empréstimos) e escreva 10 relatórios.
- Construa o banco de um marketplace (vendedores, produtos, avaliações) com particionamento por data em pedidos.
- Faça um ETL que lê 1 milhão de linhas em blocos de 5.000 e alimenta uma tabela agregada.

- Implemente auditoria automática via triggers em pelo menos três tabelas críticas.
- Escreva a paginação por keyset de uma listagem infinita (feed) e compare a performance com OFFSET usando EXPLAIN ANALYZE.

Boa jornada! SQL recompensa a prática: escreva muitas consultas, meça com EXPLAIN e leia planos de execução. Em pouco tempo você vai enxergar índices e reformulações antes mesmo de escrever a query.